



Bacharelado em Ciência da Computação
Componente: Computação Gráfica → Professor: Claudio Kleina
Aula 4 = Triângulos, Pirâmide e Normal

1 Triângulos

No universo da computação gráfica, especialmente no OpenGL, o triângulo não é apenas uma forma geométrica simples; ele é o **átomo fundamental** de toda a renderização 3D.

A importância dessa primitiva reside em três pilares: a sua natureza matemática, a eficiência de hardware e a flexibilidade na modelagem.

A Geometria Perfeita: Planaridade Garantida

O triângulo é o polígono mais simples possível no plano euclidiano, possuindo sempre três vértices. A propriedade crucial aqui é que **três pontos sempre formam um plano**.

Ao contrário de um quadrado ou qualquer polígono com quatro ou mais lados, que pode ser "deformado" (torcido) se um dos vértices for movido para fora do plano, um triângulo é intrinsecamente plano. Isso elimina ambiguidades matemáticas para o motor gráfico: não há incerteza sobre como a superfície deve ser preenchida ou como a luz deve ser calculada.

Para criar um triângulo preenchido usamos "GL_TRIANGLES":

```
glBegin(GL_TRIANGLES)
    glVertex3f( 0.0,  1.0,  0.0) # Ápice
    glVertex3f(-1.0, -1.0,  1.0) # Inferior Esquerdo
    glVertex3f( 1.0, -1.0,  1.0) # Inferior Direito
glEnd()
```

##Lembre-se que a ordem dos vértices (sentido horário ou anti-horário) define para onde a "face" do objeto está apontando (Normal), o que é crucial para cálculos futuros de iluminação.

2 Construção de uma pirâmide

Para criar uma única pirâmide usando triângulos no OpenGL, precisamos definir **4 triângulos para as faces laterais** e **2 triângulos para a base quadrada** (totalizando 6 triângulos).

#Código da Pirâmide

```
import glfw
from OpenGL.GL import *
from OpenGL.GLU import *

def desenhar_piramide():
    """
    Desenha uma pirâmide de base quadrada usando apenas triângulos.
    O ápice está em (0, 1, 0) e a base está no plano Y = -1.
    """
    glBegin(GL_TRIANGLES)

    # --- Face Frontal (Vermelha) ---
    glColor3f(1.0, 0.0, 0.0)
    glVertex3f( 0.0,  1.0,  0.0) # Ápice
    glVertex3f(-1.0, -1.0,  1.0) # Inferior Esquerdo
    glVertex3f( 1.0, -1.0,  1.0) # Inferior Direito

    # --- Face Direita (Verde) ---
    glColor3f(0.0, 1.0, 0.0)
    glVertex3f( 0.0,  1.0,  0.0)
    glVertex3f( 1.0, -1.0,  1.0)
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
# --- Face Traseira (Azul) ---
```

```
glColor3f(0.0, 0.0, 1.0)
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, -1.0)
```

```
# --- Face Esquerda (Amarela) ---
```

```
glColor3f(1.0, 1.0, 0.0)
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f(-1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, 1.0)
```

```
# --- Base Quadrada (Magenta) - Dividida em 2 triângulos ---
```

```
glColor3f(1.0, 0.0, 1.0)
```

```
# Primeiro Triângulo da Base
```

```
glVertex3f(-1.0, -1.0, 1.0)
```

```
glVertex3f( 1.0, -1.0, 1.0)
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
# Segundo Triângulo da Base
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, 1.0)
```

```
glEnd()
```

```
def main():
```

```
    if not glfw.init():
```

```
        return
```

```
janela = glfw.create_window(800, 600, "Pirâmide Única com Triângulos", None, None)
```

```
if not janela:
```

```
    glfw.terminate()
```

```
    return
```

```
glfw.make_context_current(janela)
```

```
glEnable(GL_DEPTH_TEST) # Habilita o teste de profundidade para o 3D
```

```
# Configuração da Projeção
```

```
glMatrixMode(GL_PROJECTION)
```

```
gluPerspective(45, 800/600, 0.1, 50.0)
```

```
glMatrixMode(GL_MODELVIEW)
```

```
while not glfw.window_should_close(janela):
```

```
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
    glLoadIdentity()
```

```
    # Posiciona e rotaciona a pirâmide para visualização
```

```
    glTranslatef(0.0, 0.0, -6.0)
```

```
    glRotatef(glfw.get_time() * 50, 0, 1, 0) # Rotação automática no eixo Y
```

```
    glRotatef(20, 1, 0, 0) # Inclinação fixa para ver o topo
```

```
    desenhar_piramide()
```

```
glfw.swap_buffers(janela)
glfw.poll_events()
```

```
glfw.terminate()
main()
```

3 Normal

No contexto de Computação Gráfica e OpenGL, uma **Normal** é um vetor matemático que é **perpendicular** (forma um ângulo de 90°) a uma superfície ou a um vértice. É como uma "seta invisível" que aponta para onde a face do objeto está "olhando".

Para que serve a Normal?

O OpenGL utiliza as Normais principalmente para calcular a **Iluminação**.

Sem Normais, a placa de vídeo não sabe como a luz deve bater no objeto. Imagine uma lanterna apontada para uma parede:

- Se a parede está de frente para a lanterna (Normal paralela ao raio de luz), ela fica muito clara.
- Se a parede está inclinada (Normal em ângulo), ela fica mais escura.
- Se a parede está de costas (Normal apontando para o lado oposto), ela fica preta.

Tipos de Normais

A. Normal de Face (Face Normal)

É um único vetor para toda a superfície plana. No caso do nosso **cubo**, cada uma das 6 faces tem uma normal apontando para uma direção cardinal:

- Face Frontal: (0,0,1) — Aponta para frente (eixo Z positivo).
- Face Superior: (0,1,0) — Aponta para cima (eixo Y positivo).

B. Normal de Vértice (Vertex Normal)

Usada para superfícies curvas (como esferas). Cada vértice tem sua própria normal. O OpenGL faz uma média entre elas para que a luz pareça "suave" (Smooth Shading) ao passar de uma face para outra, escondendo as arestas secas.

Em Computação Gráfica, uma **Normal** é um vetor unitário (de comprimento 1) que é **perpendicular** a uma superfície em um determinado ponto. Se imaginarmos uma face plana, a normal é uma linha reta que sai exatamente do centro dessa face formando um ângulo de **90°** em todas as direções.

- **Em superfícies planas (Faces):** A normal define a orientação da face no espaço global.
- **Em superfícies curvas (Vértices):** As normais nos vértices são usadas para calcular como a luz deve ser interpolada (suavizada) ao longo da forma, simulando uma curvatura que a geometria de polígonos não possui nativamente.

2A Importância nos Desenhos 3D

A normal é, talvez, o atributo mais importante depois da posição do vértice. Sem ela, o realismo é impossível. Sua importância divide-se em três pilares:

A. Cálculo de Iluminação (O Modelo de Sombreamento)

O OpenGL não "sabe" que um objeto é sólido. Ele usa a normal para calcular o produto escalar entre o vetor da luz e a superfície.

- **Luz de frente:** Se a luz atinge a normal em um ângulo de 0° (paralela), a face recebe brilho máximo.
- **Luz de raspão:** Quanto maior o ângulo entre a luz e a normal, mais escura a face se torna. É isso que gera as sombras próprias de um objeto, permitindo que o nosso cérebro perceba volume e profundidade.

B. Determinação de Visibilidade (Back-face Culling)

As normais ajudam a GPU a decidir o que deve ser desenhado. Se a normal de uma face está apontando para o lado oposto da câmera, o OpenGL entende que aquela face é a "parte de dentro" ou a "parte de trás" do objeto e pode descartá-la para economizar processamento.

C. Suavização de Geometria (Smooth Shading)

Através das normais, podemos fazer um objeto de poucos polígonos parecer perfeitamente liso. Ao interpolar as normais entre os vértices (como no sombreamento de *Phong* ou *Gouraud*), os reflexos de luz atravessam as quinas de forma suave, escondendo as arestas dos triângulos.

Como definir a normal no código?

No OpenGL clássico, você define a normal **antes** de definir os vértices da face. O estado da normal permanece o mesmo até que você a mude.

Python

```
glBegin(GL_QUADS)
```

```
# Definindo a Normal da Face Frontal
```

```
glNormal3f(0.0, 0.0, 1.0)
```

```
# Agora os vértices dessa face recebem essa luz
```

```
glVertex3f(-1, -1, 1)
```

```
glVertex3f( 1, -1, 1)
```

```
glVertex3f( 1,  1, 1)
```

```
glVertex3f(-1,  1, 1)
```

```
glEnd()
```

Sem a Normal, o objeto 3D parece um desenho 2D "chapado", pois não há sombras que definam sua volumetria. **É a Normal que permite ao OpenGL diferenciar o que é uma quina de uma superfície plana.**

Um erro comum é desenhar a normal apontando para **dentro** do objeto. Se isso acontecer, a face ficará escura quando iluminada por fora e brilhará quando a luz estiver dentro do cubo. A Normal deve sempre apontar para o lado "externo" da geometria.

4 Normal na nossa pirâmide

Como a pirâmide tem faces inclinadas, as normais não são retas como as de um cubo (que apontam apenas para Cima, Baixo, Frente...). Elas também estão inclinadas.

A Regra dos 90 Graus (Perpendicularidade)

A normal de cada face da pirâmide deve formar um ângulo de 90 graus com a superfície daquele triângulo.

Se a face frontal da pirâmide está inclinada para "frente e para cima", a normal dela também estará inclinada para **frente e para cima**. É essa inclinação que dirá ao OpenGL: *"Ei, se o Sol estiver bem em cima da pirâmide, esta face não deve brilhar tanto quanto o topo, porque a luz está batendo de raspão"*.

2. O Cálculo Matemático (Produto Vetorial)

Para alunos de nível superior, você pode explicar como o OpenGL (ou o programador) descobre essa normal. Como cada face é um triângulo com três vértices (A, B, C), a normal é o resultado do **Produto Vetorial** (Cross Product) **de dois vetores formados pelos lados do triângulo**:

$$\text{vec}\{n\} = (B - A) * (C - A)$$

O resultado deste cálculo é um vetor que aponta exatamente para fora da face.

3. Implementação no Código

No código da pirâmide que fizemos, para cada glBegin(GL_TRIANGLES), deveríamos fornecer a normal antes dos vértices.

Exemplo para a Face Frontal:

Se a face frontal tem os vértices baseados em (-1, -1, 1), (1, -1, 1) e o ápice em (0, 1, 0), a normal aproximada seria (0, 0.44, 0.89).

```
glBegin(GL_TRIANGLES)
```

```
# Esta normal diz que a face olha um pouco para cima (Y) e muito para frente (Z)
```

```

glNormal3f(0.0, 0.447, 0.894)
glColor3f(1.0, 0.0, 0.0)
glVertex3f( 0.0, 1.0, 0.0) # Ápice
glVertex3f(-1.0, -1.0, 1.0) # Base Esq
glVertex3f( 1.0, -1.0, 1.0) # Base Dir
glEnd()

```

O que acontece se a Normal estiver errada?

Se o aluno definir a normal da face frontal apontando para trás (Z negativo), o OpenGL vai achar que aquela face está de costas para a luz. O resultado? A face frontal ficará **preta**, mesmo que a luz esteja bem na frente dela! É um excelente teste prático para eles entenderem a importância do vetor.

Se você desenhar uma pirâmide com faces da mesma cor, sem definir as normais, ela parecerá um triângulo 2D sem volume. Ao adicionar as normais, cada face reagirá à luz de forma independente, revelando as arestas e a estrutura tridimensional da forma.

Para que a iluminação funcione corretamente em uma pirâmide, precisamos definir a **Normal** de cada face. Como a pirâmide tem faces inclinadas, as normais não apontam apenas para "frente" ou "cima", mas em uma direção diagonal perpendicular ao plano de cada triângulo.

Abaixo está o código completo. O sistema de iluminação (GL_LIGHTING) foi ativado para ver a diferença que as normais fazem: sem elas, a pirâmide seria apenas uma silhueta escura; com elas, cada face brilha de acordo com sua inclinação em relação à luz.

Por que esse código é importante para a aula?

1. **Valores das Normais:** Note que os valores (como 0.447 e 0.894) não são aleatórios. Eles são o resultado da normalização do vetor perpendicular à face inclinada. Se usássemos apenas (0, 1, 1), a luz funcionaria, mas o brilho seria matematicamente impreciso.
2. **O comando glNormal3f:** Ele deve vir **antes** dos vértices. Ele define a orientação para todos os vértices que vierem a seguir até que uma nova normal seja definida.
3. **Visualização do Volume:** Ao rodar o programa, os alunos notarão que as faces "se apagam" quando ficam de costas para a luz e brilham intensamente quando ficam de frente. Isso prova que o OpenGL está usando as normais para calcular o sombreado em tempo real.

```

import glfw
from OpenGL.GL import *
from OpenGL.GLU import *

# --- Configurações de Janela ---
LARGURA, ALTURA = 800, 600

def configurar_iluminacao():
    """ Habilita e configura a luz e o material """
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_COLOR_MATERIAL) # Faz com que o glColor seja levado em conta pela luz

    # Define a posição da luz (no alto, à direita e à frente)
    posicao_luz = [2.0, 2.0, 2.0, 1.0]
    glLightfv(GL_LIGHT0, GL_POSITION, posicao_luz)

    # Luz branca intensa
    luz_branca = [1.0, 1.0, 1.0, 1.0]
    glLightfv(GL_LIGHT0, GL_DIFFUSE, luz_branca)
    glLightfv(GL_LIGHT0, GL_SPECULAR, luz_branca)

def desenhar_piramide_com_iluminacao():
    """

```

Desenha a pirâmide definindo a Normal de cada face antes dos vértices.
Valores das normais calculados para perpendicularidade às faces.

""

```
glBegin(GL_TRIANGLES)
```

```
# FACE FRONTAL
```

```
glNormal3f(0.0, 0.447, 0.894) # Normal aponta para frente e levemente para cima
```

```
glColor3f(1.0, 0.0, 0.0) # Vermelho
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f(-1.0, -1.0, 1.0)
```

```
glVertex3f( 1.0, -1.0, 1.0)
```

```
# FACE DIREITA
```

```
glNormal3f(0.894, 0.447, 0.0) # Normal aponta para a direita e para cima
```

```
glColor3f(0.0, 1.0, 0.0) # Verde
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f( 1.0, -1.0, 1.0)
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
# FACE TRASEIRA
```

```
glNormal3f(0.0, 0.447, -0.894) # Normal aponta para trás e para cima
```

```
glColor3f(0.0, 0.0, 1.0) # Azul
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f( 1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, -1.0)
```

```
# FACE ESQUERDA
```

```
glNormal3f(-0.894, 0.447, 0.0) # Normal aponta para a esquerda e para cima
```

```
glColor3f(1.0, 1.0, 0.0) # Amarelo
```

```
glVertex3f( 0.0, 1.0, 0.0)
```

```
glVertex3f(-1.0, -1.0, -1.0)
```

```
glVertex3f(-1.0, -1.0, 1.0)
```

```
# BASE (Normal aponta para baixo)
```

```
glNormal3f(0.0, -1.0, 0.0)
```

```
glColor3f(1.0, 0.0, 1.0) # Magenta
```

```
# Triângulo 1 da base
```

```
glVertex3f(-1.0, -1.0, 1.0); glVertex3f( 1.0, -1.0, 1.0); glVertex3f( 1.0, -1.0, -1.0)
```

```
# Triângulo 2 da base
```

```
glVertex3f( 1.0, -1.0, -1.0); glVertex3f(-1.0, -1.0, -1.0); glVertex3f(-1.0, -1.0, 1.0)
```

```
glEnd()
```

```
def main():
```

```
    if not glfw.init():
```

```
        return
```

```
    janela = glfw.create_window(LARGURA, ALTURA, "Pirâmide com Normais e Iluminação", None, None)
```

```
    glfw.make_context_current(janela)
```

```
    glEnable(GL_DEPTH_TEST) # Z-buffer para não desenhar faces escondidas sobre as da frente
```

```
    configurar_iluminacao()
```

```
    # Configuração de Projeção
```

```

glMatrixMode(GL_PROJECTION)
gluPerspective(45, LARGURA/ALTURA, 0.1, 50.0)
glMatrixMode(GL_MODELVIEW)

while not glfw.window_should_close(janela):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()

    # Câmera e animação
    glTranslatef(0.0, 0.0, -6.0)
    glRotatef(glfw.get_time() * 50, 0, 1, 0)
    glRotatef(25, 1, 0, 0)

    desenhar_piramide_com_iluminacao()

    glfw.swap_buffers(janela)
    glfw.poll_events()

glfw.terminate()

if __name__ == "__main__":
    main()

```

A seguir está uma função em Python que recebe três pontos (P1, P2, P3) e retorna as componentes (nx, ny, nz) da normal:

```

import math
def calcular_normal(p1, p2, p3):
    # 1. Calcular dois vetores a partir dos vértices (Lados do triângulo)
    # Vetor A = P2 - P1
    ax, ay, az = p2[0] - p1[0], p2[1] - p1[1], p2[2] - p1[2]
    # Vetor B = P3 - P1
    bx, by, bz = p3[0] - p1[0], p3[1] - p1[1], p3[2] - p1[2]

    # 2. Produto Vetorial (Cross Product): N = A x B
    nx = (ay * bz) - (az * by)
    ny = (az * bx) - (ax * bz)
    nz = (ax * by) - (ay * bx)

    # 3. Normalização (Ajustar o comprimento para 1.0)
    comprimento = math.sqrt(nx**2 + ny**2 + nz**2)

    if comprimento != 0:
        nx /= comprimento
        ny /= comprimento
        nz /= comprimento

    return (nx, ny, nz)

```

A ordem importa! Se você calcular $\mathbf{v}\{A\} \times \mathbf{v}\{B\}$, a normal aponta para um lado. Se calcular $\mathbf{v}\{B\} \times \mathbf{v}\{A\}$, ela apontará para o lado oposto (dentro do objeto).

Resumo do Fluxo:

1. **Subtração:** $P_2 - P_1$ e $P_3 - P_1$ (cria as arestas).
2. **Multiplicação Cruzada:** Calcula o determinante (acha a perpendicular).

3. **Raiz Quadrada e Divisão:** Ajusta o tamanho para 1 (garante a precisão da luz).

Importante: Se o triângulo for muito pequeno ou muito grande, a normal calculada no Passo 2 terá tamanhos diferentes, mas após o Passo 3, ambas serão idênticas. Isso garante que a luz brilhe com a mesma intensidade em faces de tamanhos diferentes!

Código complementar da casa com controle pelo teclado

```
import glfw
from OpenGL.GL import *
from OpenGL.GLU import *

# --- Variáveis Globais de Navegação ---
rot_x = 20.0 # Rotação vertical (Cima/Baixo)
rot_y = 45.0 # Rotação horizontal (Ao redor)
distancia = -10.0 # Distância da câmera (Zoom)

def tratar_teclado(window, key, scancode, action, mods):
    global rot_x, rot_y, distancia
    passo = 2.0 # Velocidade da rotação

    if action == glfw.PRESS or action == glfw.REPEAT:
        # Setas Esquerda/Direita: Gira ao redor da casa
        if key == glfw.KEY_LEFT: rot_y -= passo
        if key == glfw.KEY_RIGHT: rot_y += passo

        # Setas Cima/Baixo: Sobe ou desce a visão
        if key == glfw.KEY_UP: rot_x -= passo
        if key == glfw.KEY_DOWN: rot_x += passo

        # Teclas W/S: Zoom (Para frente e para trás)
        if key == glfw.KEY_W: distancia += 0.5
        if key == glfw.KEY_S: distancia -= 0.5

def desenhar_cubo(largura, altura, profundidade):
    glBegin(GL_QUADS)
    # Face Frontal (Bege)
    glColor3f(0.8, 0.7, 0.5)
    glVertex3f(-largura, 0, profundidade); glVertex3f(largura, 0, profundidade)
    glVertex3f(largura, altura, profundidade); glVertex3f(-largura, altura, profundidade)
    # Face Traseira
    glVertex3f(-largura, 0, -profundidade); glVertex3f(-largura, altura, -profundidade)
    glVertex3f(largura, altura, -profundidade); glVertex3f(largura, 0, -profundidade)
    # Lateral Direita
    glColor3f(0.7, 0.6, 0.4)
    glVertex3f(largura, 0, profundidade); glVertex3f(largura, 0, -profundidade)
    glVertex3f(largura, altura, -profundidade); glVertex3f(largura, altura, profundidade)
    # Lateral Esquerda
    glVertex3f(-largura, 0, profundidade); glVertex3f(-largura, altura, profundidade)
    glVertex3f(-largura, altura, -profundidade); glVertex3f(-largura, 0, -profundidade)
    glEnd()

# Porta
glBegin(GL_QUADS)
glColor3f(0.4, 0.2, 0.0)
glVertex3f(-0.3, 0, profundidade + 0.01)
glVertex3f(0.3, 0, profundidade + 0.01)
glVertex3f(0.3, 1.2, profundidade + 0.01)
glVertex3f(-0.3, 1.2, profundidade + 0.01)
glEnd()
```

```

def desenhar_telhado(largura, altura_base, profundidade):
    altura_telhado = altura_base + 1.5
    glBegin(GL_TRIANGLES)
    glColor3f(0.6, 0.2, 0.2)
    # Frontal
    glVertex3f(-largura - 0.2, altura_base, profundidade + 0.2)
    glVertex3f(largura + 0.2, altura_base, profundidade + 0.2)
    glVertex3f(0, altura_telhado, 0)
    # Traseiro
    glVertex3f(-largura - 0.2, altura_base, -profundidade - 0.2)
    glVertex3f(0, altura_telhado, 0)
    glVertex3f(largura + 0.2, altura_base, -profundidade - 0.2)
    glEnd()

    glBegin(GL_QUADS) # Laterais do telhado
    glColor3f(0.5, 0.1, 0.1)
    glVertex3f(largura + 0.2, altura_base, profundidade + 0.2)
    glVertex3f(largura + 0.2, altura_base, -profundidade - 0.2)
    glVertex3f(0, altura_telhado, 0); glVertex3f(0, altura_telhado, 0)
    glVertex3f(-largura - 0.2, altura_base, profundidade + 0.2)
    glVertex3f(0, altura_telhado, 0); glVertex3f(0, altura_telhado, 0)
    glVertex3f(-largura - 0.2, altura_base, -profundidade - 0.2)
    glEnd()

def main():
    if not glfw.init(): return
    janela = glfw.create_window(800, 600, "Navegação na Casa 3D", None, None)
    glfw.make_context_current(janela)
    glfw.set_key_callback(janela, tratar_teclado)
    glEnable(GL_DEPTH_TEST)

    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 800/600, 0.1, 100.0)
    glMatrixMode(GL_MODELVIEW)

    while not glfw.window_should_close(janela):
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
        glLoadIdentity()

        # --- APLICAÇÃO DA NAVEGAÇÃO ---
        glTranslatef(0, -1, distancia) # Afasta/Aproxima a câmera
        glRotatef(rot_x, 1, 0, 0)    # Gira em torno do eixo X (Vertical)
        glRotatef(rot_y, 0, 1, 0)    # Gira em torno do eixo Y (Horizontal)

        desenhar_cubo(2, 2, 2)
        desenhar_telhado(2, 2, 2)

        glfw.swap_buffers(janela)
        glfw.poll_events()
    glfw.terminate()

if __name__ == "__main__":
    main()

```

Bom estudo!